LLM ENGINEERING / AGENTS / APPLIED AI

Reliable LLM systems, built for real-world use.

I am SX Qian, an LLM engineer focused on agentic workflows, retrieval systems, structured generation, model adaptation, and evaluation-driven AI products.

Multi-Agent	RAG	PyTorch	LoRA	Evaluation	Deployment

01	**Enterprise Financial Compliance Multi-Agent System**	Anonymized enterprise case study
02	**Domain-Specific RAG for Causal Inference**	Public GitHub project
03	**Qwen Vision Fine-Tuning for OCR**	Public GitHub project

ENGINEERING APPROACH

I make model behavior testable and traceable through output constraints, intermediate validation, safe execution, evaluation, and evidence-based iteration.

PROJECT 01 / ANONYMIZED ENTERPRISE CASE

Financial compliance multi-agent system

A production-oriented system that transforms complex policy documents into structured rules, executable validation logic, risk analysis, and compliance reports.

CORE CHALLENGE	The main difficulty was not simply coordinating agents. Every intermediate output needed to be structured, executable, validated, and traceable under ambiguous policy language and imperfect data mappings.

System workflow

Policy Text	Rule Agent	Rule Schema	Data Match	Validator	Risk	Report

My responsibilities

RULE STRUCTURE	Defined unified rule fields and data constraints, then converted natural-language policies into a consistent Rule Schema.
EXECUTABLE VALIDATION	Compiled structured rules into Python validators and checked generated logic with AST inspection and automated tests.
AGENT RELIABILITY	Introduced constrained decoding, schema validation, retry and recovery logic, and cross-checks between execution results and model conclusions.
EVALUATION	Designed metrics around schema validity, field mapping, unit-test success, risk detection, compliance validation, label consistency, and hallucinated fields.
SERVING	Deployed model inference with vLLM and Docker and tested stage-specific concurrency settings.

Python	Pydantic	JSON Schema	AST	vLLM	Docker

PROJECT 01 / RELIABILITY AND EVALUATION

Engineering beyond prompting

Early prompt-only control produced missing fields, malformed JSON, and inconsistent structures. The solution combined model constraints with deterministic software checks.

OUTPUT CONTRACT	JSON Schema constrained decoding and Pydantic validation established machine-checkable interfaces between agents.
FAILURE RECOVERY	Invalid outputs triggered structured retries and explicit error handling instead of silently propagating failures downstream.
CODE SAFETY	Generated validators were parsed and inspected before execution, then exercised through unit tests.
CROSS-VALIDATION	Execution results were compared against model conclusions to surface unsupported compliance judgments.

Model and dataset work

I participated in building training and evaluation data spanning policy text, structured rules, validation logic, unit tests, and risk or compliance labels. I also explored SFT, LoRA, DPO, and RFT to improve parsing accuracy, structured-generation stability, validator correctness, and label consistency.

Concurrency trade-off

Concurrency experiments improved overall throughput but also exposed occasional result drift, including a false compliance judgment when a required association key was missing. Instead of applying one global limit, concurrency was tuned by agent stage to balance speed and stability.

KEY LESSON

Enterprise agent systems become dependable when each step has an explicit contract, a verifier, an observable failure path, and an evaluation that measures more than final-answer similarity.

PROJECT 02 / PUBLIC GITHUB

Domain-Specific RAG for Causal Inference

OPEN GITHUB REPOSITORY >

A retrieval-augmented assistant adapted for causal-inference knowledge and recommendation-analysis questions. It combines domain documents with local vector retrieval before generating an answer.

PROBLEM	General-purpose language models can produce fluent explanations without grounding them in the project materials. This workflow adds a retrieval layer so responses can be conditioned on relevant domain context.

Pipeline

Domain Documents	Chunking	Embeddings	FAISS	Top-k Context	Gemini	Answer

KNOWLEDGE SOURCE	Domain-specific materials for causal inference and recommendation analysis are prepared for retrieval.
EMBEDDING LAYER	Sentence-transformer embeddings convert text chunks and user questions into vectors.
VECTOR INDEX	FAISS supports local similarity search without requiring a hosted vector database.
GENERATION	Retrieved context is assembled into a prompt and passed to Gemini for grounded response generation.
ORCHESTRATION	LangChain components connect document processing, retrieval, prompting, and answer generation.

Python	LangChain	Sentence Transformers	FAISS	Gemini

PROJECT 02 / DESIGN AND EVALUATION

What the RAG system demonstrates

The project shows how a generic RAG pipeline can be reshaped around a specific analytical domain rather than treated as a universal chatbot.

DOMAIN ADAPTATION	Retrieval content and prompts are organized around causal-inference terminology and recommendation-analysis use cases.
LOCAL RETRIEVAL	FAISS keeps the index lightweight and reproducible for experimentation.
SEPARATION OF CONCERNS	Embedding, indexing, retrieval, and generation remain distinct stages that can be inspected or replaced independently.
GROUNDED PROMPTING	The generator receives retrieved context instead of relying only on parametric model memory.

Evaluation plan

A robust evaluation for this system should separate retrieval quality from generation quality. Retrieval can be checked with relevance-oriented measures and manual inspection of returned chunks; answer quality can be assessed for groundedness, completeness, domain correctness, and unsupported claims.

RETRIEVAL CHECKS	Are the returned chunks relevant, diverse, and sufficient for the question?
ANSWER CHECKS	Does the response use the retrieved context correctly and avoid unsupported statements?
FAILURE ANALYSIS	What happens for ambiguous, out-of-domain, or retrieval-empty questions?
FUTURE EXTENSIONS	Metadata filtering, reranking, citation display, query rewriting, and a repeatable evaluation set.

ROLE RELEVANCE	This project demonstrates practical RAG architecture, local vector retrieval, domain adaptation, and the ability to reason about evaluation beyond whether an answer merely sounds plausible.

VIEW SOURCE, README, AND PROJECT FILES ON GITHUB >

PROJECT 03 / PUBLIC GITHUB

Qwen3.5 Vision Fine-Tuning for LaTeX OCR

OPEN GITHUB REPOSITORY >

An end-to-end PyTorch workflow for adapting a Qwen3.5-style vision-language model to image-to-LaTeX generation with LoRA.

PROJECT GOAL	Understand the complete multimodal fine-tuning lifecycle - data preparation, vision-text alignment, parameter-efficient adaptation, distributed training, checkpoint recovery, metrics, and attention analysis - rather than only running inference.

Training configuration

BASE MODEL	Qwen3.5-VL 4B with LoRA applied for parameter-efficient adaptation.
DATASET	unsloth/LaTeX_OCR with 68,686 image and LaTeX-text pairs.
LORA	Rank 8, alpha 16, and dropout 0.05.
COMPUTE	PyTorch Distributed Data Parallel across two NVIDIA V100 16 GB GPUs.
MEMORY STRATEGY	Gradient checkpointing, disabled use_cache, small per-GPU batches, and chunk-based training.
RECOVERY	Resume-ready checkpoints support training across multiple cluster sessions.

PyTorch	Qwen3.5-VL	LoRA	DDP	Multimodal	LaTeX OCR

PROJECT 03 / DATA, TRAINING, AND ANALYSIS

Building a reproducible multimodal pipeline

The repository separates setup, data, model, training, distributed training, evaluation, and attention analysis into reusable notebooks and Python modules.

INPUT CONSTRUCTION	Images are resized and converted to vision tokens, then aligned with prompt tokens and target LaTeX sequences.
SEQUENCE PLANNING	Token-length inspection informed 384-token limits and reduced avoidable truncation.
CHUNKED TRAINING	The dataset is processed in chunks so large-scale runs fit the available cluster memory and session constraints.
EXPERIMENT TRACKING	Metrics and artifacts are exported in JSON, CSV, and PNG formats.
EVALUATION	The pipeline evaluates unseen samples and includes attention visualization for model interpretation.
TRAINING BEHAVIOR	The public experiment notes show early loss reduction followed by stable optimization without NaN loss or exploding gradients.

Repository workflow

Setup	Data	Model	Train	DDP	Evaluate	Attention

WHAT THIS DEMONSTRATES	Hands-on multimodal model adaptation, GPU-aware training design, checkpoint-resumable experimentation, distributed PyTorch, and evaluation that goes beyond a single output example.

Engineering considerations

The project makes resource constraints explicit: sequence length, image dimensions, batch size, checkpointing, chunk size, and multi-session recovery are treated as design parameters. That is important when moving from a notebook demonstration to a repeatable training workflow.

VIEW NOTEBOOKS, TRAINING CODE, AND README ON GITHUB >

EMAIL	sxqian@bu.edu
GITHUB	github.com/sxqian-lgtm